SOLAR▸SWARM

Technical Reference
Martin Pham | Perth, AU

# Data Collection Engine

## Overview

The data collection engine runs on a 4-hour cycle (scheduler job `data_collection`). It collects fresh market data from four sources, stores raw records in `collected_data`, then the pipeline processor runs 30 minutes later to normalise, deduplicate, and route high-value signals to the message bus.

**Files:**

- `data_collection/orchestrator.py` — coordinates collection across all active sources
- `data_collection/agents/web_scraper.py` — HTML scraping
- `data_collection/agents/api_poller.py` — GHL API polling
- `data_collection/agents/social_signal.py` — social media signal detection
- `data_collection/agents/price_monitor.py` — CPL benchmark tracking
- `data_collection/pipeline/processor.py` — deduplication, enrichment, bus routing

## Collection Sources

### Source Registry

All active sources are stored in the `collection_sources` table. Each source has a `source_type` that determines which agent handles it.

| SOURCE_TYPE | AGENT | WHAT IT COLLECTS |
|---|---|---|
| `web_scrape` | `web_scraper.py` | Solar installer registries, directories |
| `api_poll` | `api_poller.py` | GHL contacts and pipeline opportunities |
| `social` | `social_signal.py` | LinkedIn buying signals |
| `price_monitor` | `price_monitor.py` | CPL benchmarks and lead price trends |

### 1. Web Scraper (`web_scraper.py`)

Scrapes public HTML sources for solar company data. Requires `requests` and `beautifulsoup4`.

`collect(source)` → `{success, records, signals, error}`

The `source` dict must include `config.url` and `config.data_type`. Supported data types:

| DATA_TYPE | PARSER | FIELDS EXTRACTED |
|---|---|---|
| `solar_installer` | `_parse_installer_table()` | company_name, licence_number, state, source_url |
| `generic` | Link extractor | url, text, source_url |

Falls back to mock data (`_mock_records()`) when requests/bs4 are unavailable or the URL is blocked. Mock data returns 3 sample installer records for development.

### 2. API Poller (`api_poller.py`)

Polls GoHighLevel REST API for contacts and pipeline opportunities. Falls back to mock data when `GHL_API_KEY` is not configured.

`collect(source)` → `{success, records, signals, error}`

The `source` dict `config.endpoint` field determines what to fetch:

| ENDPOINT | WHAT'S RETURNED |
|---|---|
| `contacts` | GHL contact records |
| `opportunities` | Pipeline opportunity records |
| Other | Raw response |

The old GHL API base URL used here is `https://rest.gohighlevel.com/v1` (note: this differs from the `ghl_client.py` which uses `https://services.leadconnectorhq.com`).

### 3. Social Signal (`social_signal.py`)

Monitors LinkedIn and other social platforms for buying signals — hiring posts, scaling announcements, and pain-point complaints that indicate a company is ready for CRM automation.

`collect(source)` → `{success, records, signals, error}`

The `source` dict `config.platform` and `config.query` fields configure what to scan.

**Signal keywords monitored:**

```
SIGNAL_KEYWORDS = [
    "hiring", "scaling", "growth", "looking for", "need help",
    "manual process", "too many leads", "follow up", "crm",
    "solar install", "new office", "expanding", "franchise",
]
```

**Classification logic:**

1. Count keyword hits in post text
2. Base signal strength: 0 hits → low, 1–2 hits → medium, 3+ hits → high
3. If OpenAI is configured and score ≥ 2: GPT-4o refines the classification with context

**Note:** Live social API access requires paid credentials (LinkedIn API, etc.). The current implementation uses mock data with realistic examples for development.

### 4. Price Monitor (`data_collection/agents/price_monitor.py`)

Tracks CPL (cost per lead) benchmarks and lead price trends in the Australian solar market. Results are stored in the `time_series` table for trend detection.

---

## Pipeline Processor (`processor.py`)

`process_batch(since_minutes=60)` → `{processed, deduplicated, signals_posted}`

The processor runs every 4 hours (30 minutes after the collection cycle) and processes all unprocessed records from `collected_data`.

### Step 1: Fetch Unprocessed Records

```
SELECT * FROM collected_data
WHERE pipeline_processed IS NULL
AND collected_at >= datetime('now', '-{since_minutes} minutes')
ORDER BY collected_at ASC
LIMIT 200
```

### Step 2: Deduplication

Each record gets a fingerprint from key fields:

```
fingerprint = "|".join([
    data.get("company_name", "").lower(),
    data.get("url", "").lower(),
    data.get("keyword", "").lower(),
    data.get("id", "").lower(),
])
```

Records with the same fingerprint in the current batch are skipped.

### Step 3: Enrichment Scoring

| SOURCE_TYPE | CONDITION | ENRICHED_SCORE |
|---|---|---|
| `social` | signal_strength == "high" | 9 |
| `social` | signal_strength == "medium" | 6 |
| `web_scrape` | company_name present | 5 |
| `api_poll` | any | 7 |

### Step 4: Signal Extraction

Records with `enriched_score ≥ 5` are converted to signals:

| SOURCE_TYPE | SIGNAL_TYPE | FIELDS |
|---|---|---|
| `social` | `social_buying_signal` | company, strength, evidence, url |
| `web_scrape` | `new_prospect_found` | company, state, licence |
| `api_poll` | `ghl_contact_update` | contact_id, name |

### Step 5: Bus Routing

Signals with score ≥ 5 are posted to `research_queue` on the message bus:

```
message_bus.post(
    from_agent="pipeline_processor",
    to_queue="research_queue",
    msg_type="ALERT",
    payload={"signal": signal, "source_record_id": rec["record_id"]},
    priority="HIGH" if score >= 8 else "NORMAL",
)
```

### Step 6: Mark Processed

Each processed record gets `pipeline_processed` stamped with the current timestamp.

---

## How to Add a New Collection Source

1. **Create an agent file** in `data_collection/agents/your_source.py`

Implement a `collect(source: dict) -> dict` function that:

- Reads config from `source["config"]`
- Returns `{success: bool, records: int, signals: int}`
- Writes records to `collected_data` table

1. **Register the source type** in `data_collection/orchestrator.py`

Add your `source_type` to the dispatcher dict that maps types to agent modules.

1. **Insert a row** into `collection_sources` table:

` python

from memory.database import insert, json_payload

insert("collection_sources", {

"source_id": "my_source_001",

```
"name": "My Data Source",
"source_type": "your_type", # must match your agent's handling
"url_template": "https://...",
"config": json_payload({"url": "https://...", "data_type": "my_type"}),
"collection_frequency": "daily",
"frequency_hours": 24,
"priority": "NORMAL",
"active": 1,
})
`
```

---

## `collected_data` Table Schema

| COLUMN | TYPE | DESCRIPTION |
|---|---|---|
| `id` | INTEGER PK | Auto-increment |
| `collected_at` | TEXT | Timestamp (UTC) |
| `source_id` | TEXT NOT NULL | References collection_sources.source_id |
| `source_type` | TEXT | web_scrape / api_poll / social / price_monitor |
| `record_type` | TEXT NOT NULL | Data category (solar_installer, contact, etc.) |
| `record_key` | TEXT | Optional dedup key |
| `raw_data` | TEXT | Original JSON from source |
| `normalized_data` | TEXT | Processed/normalised JSON |
| `quality_score` | REAL | 0.0–1.0 quality rating |
| `dedup_hash` | TEXT | Fingerprint for deduplication |
| `processed` | INTEGER | 0/1 — has been through pipeline |
| `normalized` | INTEGER | 0/1 — has been normalised |

**Note:** The `pipeline_processor.py` also uses a `pipeline_processed` column (added via migration) to track which records have been through the pipeline processor specifically.

---

## Processing Frequency

| JOB | FREQUENCY | OFFSET |
|---|---|---|
| Data collection | Every 4 hours | Starts immediately |
| Pipeline processor | Every 4 hours | Starts 30 minutes after collection |
| Scout agent | Daily 08:00 UTC | Reads from collected_data (last 2 days) |
| Research engine | Daily 06:00 UTC | Processes research_queue messages from bus |

The 30-minute offset between collection and processing ensures fresh records are available before processing begins.